Practical 1

Part A: Aim: To implement an Inverted Index construction algorithm to facilitate efficient document retrieval based on keyword searches.

Implementation Code (Python)

```python
# AIM: Implement an inverted index construction algorithm.
import re
from collections import defaultdict

class InvertedIndex:
    def __init__(self):
        self.index = defaultdict(set) # token -> set of doc_ids

    def tokenize(self, text):
        return re.findall(r'\b\w+\b', text.lower())

    def add_document(self, doc_id, text):
        tokens = self.tokenize(text)
        for token in tokens:
            self.index[token].add(doc_id)

    def display(self):
        print("Inverted Index:")
        for token, doc_ids in sorted(self.index.items()):
            print(f"'{token}': {sorted(list(doc_ids))}")

if __name__ == "__main__":
    idx = InvertedIndex()
    idx.add_document(1, "This is the first document")
    idx.add_document(2, "This is the second document")
    idx.add_document(3, "Another document is here")
    idx.display()
```

Output:

```
(venv) PS C:\Users\tembu\OneDrive\Desktop\IR> & C:,
{1, 2, 3}
(venv) PS C:\Users\tembu\OneDrive\Desktop\IR>
```

Part B: Aim: Document Indexing and Retrieval (Part B) – Build a simple document retrieval system using the constructed index.

Code:

```python
class DocumentRetrievalSystem:
    def __init__(self, index):
        self.index = index

    def search(self, query):
        # Preprocess query by lowercasing and splitting into individual
terms
        terms = query.lower().split()
        relevant_docs = set()

        for term in terms:
            if term in self.index:
                # Add document IDs associated with the term to the results
set
                relevant_docs.update(self.index[term])

        return relevant_docs

# --- Example Usage ---
if __name__ == "__main__":
    # Manually defined index for demonstration
    index = {
        'apple': {1, 2, 3},
        'banana': {2, 3, 4},
        'orange': {3, 4, 5}
    }

    retrieval_system = DocumentRetrievalSystem(index)

    query = "apple orange"
    relevant_docs = retrieval_system.search(query)

    print("Relevant Documents for query '{}' are:".format(query))
    print(relevant_docs)  # Output: {1, 2, 3, 4, 5}
```

Output:

```
(venv) PS C:\Users\tembu\OneDrive\Desktop\IR> & C:
Relevant Documents for query 'apple orange' are:
{1, 2, 3, 4, 5}
(venv) PS C:\Users\tembu\OneDrive\Desktop\IR>
```

Practical 2

Part A: Aim: Retrieval Models – Implement the Boolean retrieval model and process queries.

Code:

```
class BooleanRetrieval:
    def __init__(self, index):
        self.index = index

    def boolean_search(self, query):
        query = query.lower()
        terms = query.split()

        # Handle AND logic: find common documents between terms
        if 'and' in terms:
            intersection = None
            for term in terms:
                if term != 'and':
                    if intersection is None:
                        intersection = self.index.get(term, set())
                    else:
                        intersection &= self.index.get(term, set())
            return intersection if intersection is not None else set()

        # Handle OR logic: combine all documents from terms
        elif 'or' in terms:
            union = set()
            for term in terms:
                if term != 'or':
                    union |= self.index.get(term, set())
            return union

        # Handle NOT logic: find documents that do not contain the term
        elif 'not' in terms:
            # Assuming all_docs is the set of all unique document IDs in the
index
            all_docs = set().union(*self.index.values())
            complement = all_docs
            for term in terms:
                if term != 'not':
                    complement = complement - self.index.get(term, set())
            return complement

        # Simple single term search
        else:
            return self.index.get(query, set())

# --- Example Usage ---
if __name__ == "__main__":
    index = {
        'apple': {1, 2, 3},
        'banana': {2, 3, 4},
        'orange': {3, 4, 5}
    }
```

```
    boolean_retrieval = BooleanRetrieval(index)

    print("Search 'apple and banana':",
boolean_retrieval.boolean_search("apple and banana"))
    print("Search 'apple or banana':",
boolean_retrieval.boolean_search("apple or banana"))
    print("Search 'not orange':", boolean_retrieval.boolean_search("not
orange"))
```

Part B: aim: Retrieval Models – Implement the vector space model with TF-IDF weighting and cosine similarity.

code:

```
import math

class VectorSpaceModel:
    def __init__(self, documents):
        self.documents = documents
        self.doc_vectors = self._calculate_tf_idf()

    def _calculate_tf_idf(self):
        tf_idf = {}
        total_docs = len(self.documents)

        # Calculate TF for each term in each document
        for doc_id, doc in self.documents.items():
            tf_idf[doc_id] = {}
            total_terms = len(doc)
            for term, freq in doc.items():
                # Term Frequency (TF)
                tf = freq / total_terms

                # Inverse Document Frequency (IDF)
                doc_freq = sum(1 for document in self.documents.values() if
term in document)
                idf = math.log(total_docs / (1 + doc_freq))

                # TF-IDF calculation
                tf_idf[doc_id][term] = tf * idf
        return tf_idf

    def _normalize_vector(self, vector):
        """Normalizes the vector to unit length."""
        denominator = sum(value ** 2 for value in vector.values())
        if denominator == 0:
            return vector
        norm = math.sqrt(denominator)
        return {term: value / norm for term, value in vector.items()}

    def _dot_product(self, vec1, vec2):
        """Calculates the dot product of two sparse vectors."""
        return sum(vec1.get(term, 0) * vec2.get(term, 0) for term in
set(vec1) | set(vec2))

    def cosine_similarity(self, query):
        query_vector = {}
        # Calculate TF for query terms
```

```
        total_terms = len(query)
        total_docs = len(self.documents)

        for term, freq in query.items():
            tf = freq / total_terms

            # Calculate IDF for query terms based on document collection
            doc_freq = sum(1 for document in self.documents.values() if term
in document)
            idf = math.log(total_docs / (1 + doc_freq))

            query_vector[term] = tf * idf

        # Normalize query vector
        query_vector = self._normalize_vector(query_vector)

        # Compute cosine similarity between query and documents
        similarities = {}
        for doc_id, doc_vector in self.doc_vectors.items():
            normalized_doc_vec = self._normalize_vector(doc_vector)
            similarities[doc_id] = self._dot_product(query_vector,
normalized_doc_vec)

        return similarities

# --- Example Usage ---
if __name__ == "__main__":
    # Documents represented as term-frequency dictionaries
    documents = {
        1: {'apple': 2, 'banana': 3},
        2: {'apple': 1, 'banana': 2, 'orange': 1},
        3: {'banana': 3, 'orange': 2}
    }

    vsm = VectorSpaceModel(documents)

    # Query represented as a term-frequency dictionary
    query = {'apple': 1, 'banana': 1}

    similarities = vsm.cosine_similarity(query)

    print("Cosine Similarities:")
    for doc_id, score in similarities.items():
        print(f"Document {doc_id}: {score:.4f}")
```

Output:

```
(venv) PS C:\Users\tembu\OneDrive\Desktop\IR> & C:/Users/tembu/OneDrive/Des
{1: 0.43152310867767135, 2: 0.19178804830118726, 3: 0.43152310867767135}
(venv) PS C:\Users\tembu\OneDrive\Desktop\IR>
```

Practical 3

Part A: aim: Spelling Correction in IR Systems – Develop a spelling correction module using edit distance algorithms.

code:

```
class SpellingCorrector:
    def __init__(self, vocabulary):
        self.vocabulary = vocabulary

    def _edit_distance(self, word1, word2):
        """Calculates the Levenshtein edit distance between two words."""
        m, n = len(word1), len(word2)
        # Create a matrix to store distances of subproblems
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        # Base cases: transforming a string into an empty string
        for i in range(m + 1):
            dp[i][0] = i
        for j in range(n + 1):
            dp[0][j] = j

        # Fill the DP table
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = 1 + min(dp[i - 1][j],      # Deletion
                                       dp[i][j - 1],      # Insertion
                                       dp[i - 1][j - 1])  # Substitution
        return dp[m][n]

    def correct_spelling(self, word):
        """Finds the word in vocabulary with the minimum edit distance."""
        min_distance = float('inf')
        corrected_word = word

        for vocab_word in self.vocabulary:
            distance = self._edit_distance(word, vocab_word)
            if distance < min_distance:
                min_distance = distance
                corrected_word = vocab_word

        return corrected_word

# --- Example Usage ---
if __name__ == "__main__":
    vocabulary = {'apple', 'banana', 'orange', 'grape', 'kiwi'}
    corrector = SpellingCorrector(vocabulary)

    print(f"Input: 'appl' -> Correction:
{corrector.correct_spelling('appl')}")
    print(f"Input: 'orang' -> Correction:
{corrector.correct_spelling('orang')}")
    print(f"Input: 'banama' -> Correction:
{corrector.correct_spelling('banama')}")
```

Output:

```
(venv) PS C:\Users\tembu\OneDrive\Desktop\IR> & C:/Users/tem
apple
orange
banana
(venv) PS C:\Users\tembu\OneDrive\Desktop\IR>
```

Part B: aim: Spelling Correction in IR Systems – Integrate the spelling correction module into an information retrieval system.

code:

```
from spellchecker import SpellChecker

class InformationRetrievalSystem:
    def __init__(self):
        # Initialize the spell checker (requires 'pyspellchecker' library)
        self.spell_checker = SpellChecker()

    def preprocess_query(self, query):
        """Tokenize the query by splitting on whitespace."""
        return query.split()

    def correct_spelling(self, query_tokens):
        """Identifies misspelled words and replaces them with the most
likely correction."""
        corrected_query = []
        for word in query_tokens:
            # Check if the word is in the dictionary
            if self.spell_checker.unknown([word]):
                # Fetch the best suggestion
                corrected_word = self.spell_checker.correction(word)
                corrected_query.append(corrected_word)
            else:
                corrected_query.append(word)
        return corrected_query

    def retrieve_information(self, query):
        """Main pipeline: Preprocess -> Correct -> Retrieve."""
        processed_query = self.preprocess_query(query)
        corrected_query = self.correct_spelling(processed_query)

        # In a full system, the corrected_query would be passed to the
search index
        print("Original Query: ", query)
        print("Corrected Query:", " ".join(corrected_query))

# --- Example Usage ---
if __name__ == "__main__":
    retrieval_system = InformationRetrievalSystem()

    # Example with multiple typos
    user_query = "retrive inforamtion abuot spelng correctin"
    retrieval_system.retrieve_information(user_query)
```

Practical 4

Part A: aim: Evaluation Metrics for IR Systems – Calculate precision, recall, and F-measure for a given set of retrieval results.

code:

```
def calculate_precision(retrieved_docs, relevant_docs):
    """Precision = (Relevant documents retrieved) / (Total documents
retrieved)"""
    intersection = retrieved_docs.intersection(relevant_docs)
    return len(intersection) / len(retrieved_docs) if len(retrieved_docs) >
0 else 0

def calculate_recall(retrieved_docs, relevant_docs):
    """Recall = (Relevant documents retrieved) / (Total relevant
documents)"""
    intersection = retrieved_docs.intersection(relevant_docs)
    return len(intersection) / len(relevant_docs) if len(relevant_docs) > 0
else 0

def calculate_f_measure(precision, recall):
    """F-Measure = 2 * (Precision * Recall) / (Precision + Recall)"""
    return (2 * precision * recall) / (precision + recall) if (precision +
recall) > 0 else 0

# --- Example Usage ---
if __name__ == "__main__":
    # Sets of document IDs
    retrieved_docs = {1, 2, 3, 4}
    relevant_docs = {2, 3, 5}

    precision = calculate_precision(retrieved_docs, relevant_docs)
    recall = calculate_recall(retrieved_docs, relevant_docs)
    f_measure = calculate_f_measure(precision, recall)

    print(f"Retrieved Documents: {retrieved_docs}")
    print(f"Relevant Documents:  {relevant_docs}")
    print("-" * 30)
    print(f"Precision: {precision:.2f}")
    print(f"Recall:    {recall:.2f}")
    print(f"F-measure: {f_measure:.2f}")
```

Output:

Part B: aim: Use an evaluation toolkit to measure average precision and other evaluation metrics.

code:

```
from sklearn.metrics import adjusted_rand_score,
normalized_mutual_info_score, fowlkes_mallows_score

# Assuming you have ground truth labels for your data (True categories)
ground_truth_labels = [0, 1, 1, 0, 2, 2, 2, 3, 3, 3]

# Assuming you have obtained cluster labels from your clustering algorithm
cluster_labels = [1, 0, 0, 1, 2, 2, 2, 3, 3, 3]

# Compute evaluation metrics using sklearn toolkit
# ARI: Measures the similarity between two assignments
ari = adjusted_rand_score(ground_truth_labels, cluster_labels)

# NMI: Normalization of the Mutual Information score to scale the results
between 0 and 1
nmi = normalized_mutual_info_score(ground_truth_labels, cluster_labels)

# FMI: Defined as the geometric mean of pairwise precision and recall
fmi = fowlkes_mallows_score(ground_truth_labels, cluster_labels)

# Print evaluation metrics
print("Adjusted Rand Index (ARI):", ari)
print("Normalized Mutual Information (NMI):", nmi)
print("Fowlkes-Mallows Index (FMI):", fmi)
```

Output:

Practical 5

Part a: aim: Text Categorization – Implement a text classification algorithm using Naive Bayes.

code:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Sample training data (Text and corresponding labels)
train_texts = ["I love this movie", "This movie is great", "I hate this
movie", "This movie is terrible"]
train_labels = ["positive", "positive", "negative", "negative"]

# Sample test data for prediction
test_texts = ["I like this movie", "This movie is awful"]

# Create a pipeline:
# 1. CountVectorizer: Converts text into a matrix of token counts
# 2. MultinomialNB: The Naive Bayes classifier suitable for discrete
features
model = make_pipeline(CountVectorizer(), MultinomialNB())

# Split the data into training and testing sets (using 20% for testing)
X_train, X_test, y_train, y_test = train_test_split(train_texts,
train_labels, test_size=0.2, random_state=42)

# Train the model using the training data
model.fit(X_train, y_train)

# Evaluate the model on the test split
predictions = model.predict(X_test)

# Calculate and print performance metrics
accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, predictions))

# Use the model to predict categories for the new test data
new_predictions = model.predict(test_texts)
print("\nPredictions for new test data:")
for text, label in zip(test_texts, new_predictions):
    print(f"'{text}' -> {label}")
```

Output:

Part b: aim: Text Categorization – Train the classifier on a labelled dataset and evaluate its performance.

code:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Sample labeled dataset (Expand this list for better model performance)
documents = ["I love this movie", "This movie is great", "I hate this
movie", "This movie is terrible"]
labels = ["positive", "positive", "negative", "negative"]

# Split the dataset into training and testing sets (80% train, 20% test)
# random_state ensures reproducibility of the split
X_train, X_test, y_train, y_test = train_test_split(documents, labels,
test_size=0.2, random_state=42)

# Create a pipeline that combines:
# 1. Feature extraction (CountVectorizer)
# 2. Classification algorithm (MultinomialNB)
model = make_pipeline(CountVectorizer(), MultinomialNB())

# Train (fit) the classifier on the training set
model.fit(X_train, y_train)

# Predict labels for the unseen test set
predictions = model.predict(X_test)

# Evaluation: Calculate accuracy score
accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", accuracy)

# Evaluation: Print detailed classification report (Precision, Recall,
F1-score)
print("\nClassification Report:")
print(classification_report(y_test, predictions))
```

Output:

Practical 6

Part a: aim: Clustering for Information Retrieval – Implement a clustering algorithm (e.g., K-means).

code:

```
import numpy as np

class KMeans:
    def __init__(self, k, max_iterations=100):
        self.k = k
        self.max_iterations = max_iterations

    def fit(self, X):
        # Initialize centroids randomly by picking k points from the dataset
        self.centroids = X[np.random.choice(X.shape[0], self.k, replace=False)]

        for _ in range(self.max_iterations):
            # Step 1: Assign each data point to the nearest centroid
            # np.newaxis allows broadcasting to calculate distances efficiently
            distances = np.linalg.norm(X[:, np.newaxis] - self.centroids, axis=2)
            labels = np.argmin(distances, axis=1)

            # Step 2: Update centroids based on the mean of the data points in each
cluster
            new_centroids = np.array([X[labels == i].mean(axis=0) for i in
range(self.k)])

            # Step 3: Check for convergence (if centroids stop moving)
            if np.allclose(self.centroids, new_centroids):
                break
            self.centroids = new_centroids

        self.labels = labels
        return self.labels

# --- Example Usage ---
if __name__ == "__main__":
    # Generate sample 2D data
    np.random.seed(42)
    X = np.random.rand(100, 2)

    # Instantiate and fit KMeans with 3 clusters
    kmeans = KMeans(k=3)
    labels = kmeans.fit(X)

    # Print results
    print("Cluster centers:")
    print(kmeans.centroids)
    print("\nCluster labels for first 10 points:")
    print(labels[:10])
```

Output:

Part B: aim: Apply the clustering algorithm to a set of documents and evaluate the clustering results.

code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.datasets import fetch_20newsgroups

# Step 1: Prepare Your Document Data
# Load the 20 Newsgroups dataset (a collection of approximately 20,000
newsgroup documents)
data = fetch_20newsgroups(subset='all', shuffle=True, random_state=42)

# Step 2: Vectorize the Documents
# Convert text documents into a matrix of TF-IDF features, removing common
English stop words
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(data.data)

# Step 3: Choose a Clustering Algorithm
# We use K-means with 20 clusters to match the 20 newsgroup categories
num_clusters = 20
kmeans = KMeans(n_clusters=num_clusters, random_state=42)

# Step 4: Train the Clustering Algorithm
# Fit the model to the TF-IDF matrix
kmeans.fit(X)

# Step 5: Evaluate the Clustering Results
# Silhouette score measures how similar an object is to its own cluster
compared to other clusters
# Values range from -1 to 1; higher values indicate better-defined clusters
silhouette_avg = silhouette_score(X, kmeans.labels_)
print(f"Silhouette Score: {silhouette_avg:.4f}")
```

Output: